

VectorFusion

Text-to-SVG by Abstracting Pixel-Based Diffusion Models

STUDENT 원지우

AUTHOR Ajay Jain, Amber Xie, Pieter Abbeel | UC Berkeley | CVPR 2023

Table of Contents

I	Motivation & Overview	3
II	Background: SVG & Diffusion Models	4
III	Background: Score Distillation Sampling	6
IV	Method: VectorFusion	7
V	Experiments & Results	11
VI	Discussion	13

Abstract

- Diffusion 모델은 raster 이미지 생성에 뛰어나지만, 디자이너들은 **SVG(Scalable Vector Graphics)** 형식을 선호
- SVG는 임의 해상도로 확대 가능하고 용량이 작음 (수십 KB)

VectorFusion: pixel 기반 diffusion 모델을 SVG 생성에 활용하는 방법

- captioned SVG 데이터셋 없이 동작
- differentiable vector graphics rasterizer를 최적화
- **Score Distillation Sampling (SDS)** 을 통해 pretrained diffusion model에서 semantic knowledge 증류
- raster 이미지 샘플로 초기화하여 속도와 품질 향상
- 지원 스타일: iconography, pixel art, sketch/line drawing

Introduction

- SVG는 Bézier curves, polygons, circles, lines, background colors 등 primitives로 구성
- 기존 diffusion model은 web-scale 데이터가 raster 이미지이므로 raster만 생성 가능

Existing baseline : 1) Stable Diffusion으로 text to image 생성 → LIVE로 image to SVG 변환

문제 : 복잡한 이미지는 simple vector로 표현하기 어렵, caption과 incoherent해짐, detail을 잃음

해결 : Incorporate the pretrained text-to-image diffusion model in a optimization loop

- > combines differentiable vector graphics renderer -> SDS를 차용! (iteratively refine shape parameters)
- > diffusion sampling이 optimization문제로 전환됨

Contribution

- ① SDS를 latent diffusion model(Stable Diffusion)로 확장
- ② 초기화·정규화 기법으로 품질·효율 향상
- ③ Iconography / Pixel Art / Sketch 세 가지 스타일 지원

Related Work

CLIP 기반 Vector Graphics 생성

VectorAscent, CLIPDraw: CLIP image-text similarity를 최적화하여 text prompt에서 vector 생성

StyleCLIPDraw: CLIPDraw에 VGG16 auxiliary style loss 추가 (style 유사 이미지와 feature map 차이)

Arnheim: SVG path를 neural network로 parameterize

CLIP-CLOP: evolutionary approach로 image collage 생성

→ VectorFusion은 discriminative가 아닌 **generative model** 사용

Text-to-Image 생성

DALL-E 2, Imagen, **Stable Diffusion (SD)**: latent diffusion 기반

-> 이 모델들은 직접 vector graphics로 변환 불가

Related Work

Image Vectorization

LIVE (Layer-wise Image Vectorization): differentiable rasterizer DiffVG로 closed Bézier path 반복 최적화
VectorFusion은 LIVE를 확장

Inverse Graphics with Diffusion Models

Inpainting: 이미지 일부 → 나머지 픽셀 (diffusion (zero-shot)이 prior)

DDPM-PnP: 조건 신호 → 이미지/마스크 (diffusion as plug-and-play prior)

DreamFusion: 텍스트 → 3D 파라미터 (2D diffusion (SDS)이 prior)

VectorFusion: 텍스트 → SVG 파라미터 (latent diffusion (LSDS)이 prior)

SVG & Differentiable Rendering (DiffVG)

SVG란?

이미지가 아닌 2D 도형을 그리는 **명령어 시퀀스** → 해상도 무관, 이산+연속 혼합 구조

명령어 집합: [moveTo, lineTo, cubicBezier, EOS] | 최대 6-dim 인자 (padding + mask)

Style → Primitive 매핑

- Iconography → closed Bézier curves (64 paths, 4 segments)
- Pixel Art → square polygons (32×32 grid)
- Sketch → unclosed Bézier (16 strokes, 6px fixed width)

DiffVG — 핵심 역할

Differentiable rasterizer

SVG path 파라미터 θ 에 대한 rendered image의 gradient 계산

End-to-end 최적화 가능:

Loss (image space)

→ backprop

→ control points·colors 업데이트

사용처: LIVE, CLIPDraw,

VectorFusion 모두 DiffVG 기반

Diffusion Models

DDPM Training Objective

$$\mathcal{L}_{\text{DDPM}}(\phi, \mathbf{x}) = \mathbb{E}_{t, \varepsilon} [\omega(t) \|\varepsilon_\phi(\alpha_t \mathbf{x} + \sigma_t \varepsilon) - \varepsilon\|_2^2]$$

DDIM Sampling

$$\hat{\mathbf{x}} = \frac{\mathbf{x}_t - \sigma_t \varepsilon_\phi(\mathbf{x}_t)}{\alpha_t}, \quad \mathbf{x}_{t-1} = \alpha_{t-1} \hat{\mathbf{x}} + \sigma_{t-1} \varepsilon_\phi(\mathbf{x}_t)$$

Classifier-Free Guidance

$$\hat{\varepsilon}_\phi(\mathbf{x}, y) = (1 + \omega) \cdot \varepsilon_\phi(\mathbf{x}, y) - \omega \cdot \varepsilon_\phi(\mathbf{x})$$

Stable Diffusion (Latent Diffusion)

512×512 → 64×64, 4-ch latent (VQGAN autoencoder) | T=1000 timesteps | $t \sim U(50, 950)$

SDS (DreamFusion, Poole et al. 2022)

핵심 아이디어

Frozen pretrained diffusion model p_ϕ 를 **loss function**으로 사용

→ 어떤 differentiable image parameterization $g(\theta)$ 도 SVG 포함 최적화 가능

→ θ 에 대한 prior 불필요, pixel-space prior p_ϕ 만 있으면 됨

SDS Gradient (practical form)

$$\mathcal{L}_{\text{SDS}} = \mathbb{E}_{t, \epsilon} [\sigma_t / \alpha_t w(t) \text{KL}(q(\mathbf{x}_t | g(\theta); y, t) || p_\phi(\mathbf{x}_t; y, t))]$$

$$\nabla_\theta \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t, \epsilon} [w(t) (\hat{\epsilon}_\phi(\mathbf{x}_t; y, t) - \epsilon) \frac{\partial \mathbf{x}}{\partial \theta}]$$

VectorFusion에서의 역할

p_ϕ (Teacher): frozen Stable Diffusion | **$g(\theta)$:** DiffVG rasterizer <- SVG path params | **SVG 학습 데이터 불필요**

2-Stage baseline

(a) Text $\rightarrow$ image (b) image $\rightarrow$ vector

(a) Stable Diffusion Sampling

50 steps, guidance $\omega=7.5$ | **Prompt suffix:** "minimal flat 2d vector icon. lineal color. on a white background."

K=4 샘플 생성 $\rightarrow$ CLIP ViT-B/16으로 caption과 가장 consistent한 샘플 선택 (**rejection sampling**)

(b) DiffVG + LIVE $\rightarrow$ Latent SDS Fine-tuning

1개의 path $\rightarrow$ 이미지의 한 feature만 담당하도록!

L2+UDF loss로 raster $\rightarrow$ vector 근사

LSDS loss + path reinitialization으로 caption coherence 향상

(b) DiffVG + LIVE

UDF Reconstruction Loss (LIVE)

$$\mathcal{L}_{\text{UDF}} = \frac{1}{3} \sum_{i=1}^{w \times h} d'_i \sum_{c=1}^3 (I_{i,c} - \hat{I}_{i,c})^2$$

d'_i = unsigned distance to nearest path boundary | 거리 가중 L2 → 각 path가 하나의 feature만 설명하도록 유도

Self-Intersection Regularizer

$$\mathcal{L}_{\text{Xing}} = D_1(\text{ReLU}(-D_2)) + (1 - D_1)(\text{ReLU}(D_2)).$$

D1: cubic Bézier 두 segment 간 각도의 characteristic (볼록/오목) | D2: $\sin(\alpha)$ → path 자기교차 방지

path 추가 방식

8개씩 단계적으로 추가하면서 loss 높은 영역(= 아직 못 그린 곳)을 채워 나가 총 64개가 될 때까지 반복

-> Path parameter θ 가 계속 늘어남

(b) DiffVG + LIVE

Self-Intersection Regularizer

$$\mathcal{L}_{\text{Xing}} = D_1(\text{ReLU}(-D_2)) + (1 - D_1)(\text{ReLU}(D_2)),$$

D1

$$D_1 = \mathbb{I}(\vec{AB} \times \vec{BC})$$

$\mathbb{I}$ 는 indicator function (지시함수)

- 괄호 안 조건이 참이면 1, 거짓이면 0 반환
- 즉 외적 값이 양수면 $D_1 = 1$, 음수면 $D_1 = 0$

D2

$$D_2 = \frac{\vec{AB} \times \vec{CD}}{\|\vec{AB}\| \|\vec{CD}\|}$$

$= \sin(\theta) \rightarrow D_2 > 0$: 각도가 180도 이하, $D_2 < 0$: 각도가 180도 이상
 $\rightarrow$ 외적 방향에 따라 180도 기준 교차여부가 반대로 바뀐!

(b) DiffVG + LIVE

Self-Intersection Regularizer

$$\mathcal{L}_{\text{Xing}} = D_1(\text{ReLU}(-D_2)) + (1 - D_1)(\text{ReLU}(D_2)),$$

D1	D2	Loss	교차 여부
1 ($AB \times BC > 0$)	> 0 ($\theta < 180^\circ$)	0	O
1 ($AB \times BC > 0$)	< 0 ($\theta \geq 180^\circ$)	> 0	X
0 ($AB \times BC \leq 0$)	> 0 ($\theta < 180^\circ$)	> 0	X
0 ($AB \times BC \leq 0$)	< 0 ($\theta \geq 180^\circ$)	0	O

⇒ 교차 가능 시에만 패널티 부과 가능해짐!

(b) DiffVG + LIVE



(a) Sample raster image with Stable Diffusion



(b) Convert raster image to a vector

과정

- ① 무작위로 배치된 SVG선 $\rightarrow$ DiffVG $\rightarrow$ 이미지 픽셀
- ② 이미지 픽셀 $\leftrightarrow$ 원본 이미지 Loss 구하기 (with LIVE)
- ③ DiffVG가 미리 계산해둔 미분값 $\rightarrow$ 오차 감소 방향으로 SVG선 좌표 미세 역이동

문제

LIVE는 text y 를 전혀 고려 x , pixel만 비슷하게 베끼려고 함 $\Rightarrow$ 정보 손실, text 의미와 어우러지지 x

Latent Score Distillation Sampling (LSDS)

Per-Iteration Optimization Loop

- ① DiffVG render 600×600 → ② Augment (perspective + crop) 512×512 → ③ Encode $z = E_\phi(x_{\text{aug}})$
 ④ Add noise $z_t = \alpha_t \cdot z + \sigma_t \cdot \epsilon$ → ⑤ Denoise $\hat{e}_\phi(z_t, y)$ → ⑥ Backprop LSDS (+Lxing) → update θ

LSDS Gradient

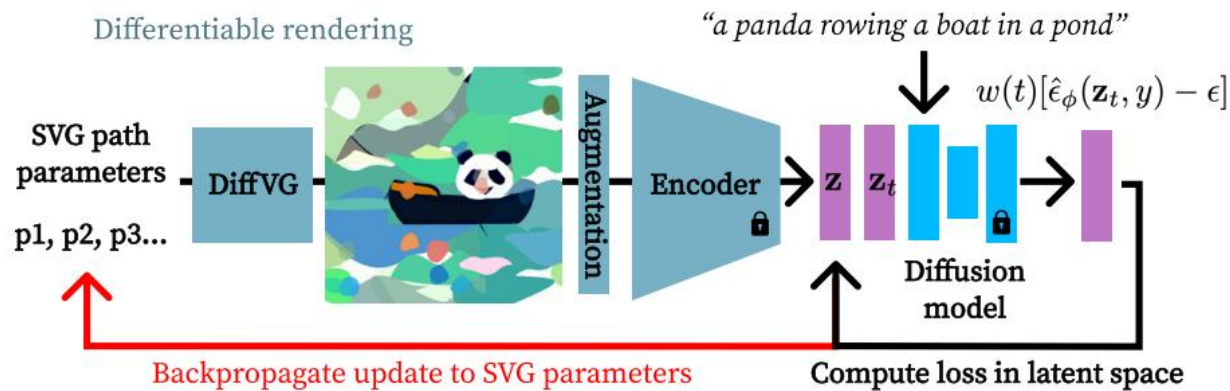
$$\nabla_{\theta} \mathcal{L}_{\text{LSDS}} = \mathbb{E}_{t, \epsilon} \left[w(t) (\hat{e}_{\phi}(\alpha_t \mathbf{z}_t + \sigma_t \epsilon, y) - \epsilon) \frac{\partial \mathbf{z}}{\partial \mathbf{x}_{\text{aug}}} \frac{\partial \mathbf{x}_{\text{aug}}}{\partial \theta} \right]$$

구현 세부사항

Timestep: $t \sim U(50, 950) / T=1000$ | **Precision:** Diffusion FP16, Jacobian $\partial z / \partial x_{\text{aug}}$ FP32 (numerical stability)

Regularization: L_Xing self-intersection penalty | **Path reinitialization:** opacity/area 임계값 이하 path → 기존 path 위에 random 위치·색상의 circle로 재초기화

Latent Score Distillation Sampling (LSDS)



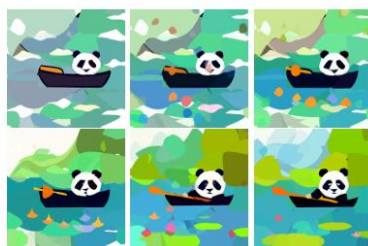
A panda rowing a boat in a pond.



(a) Sample raster image with Stable Diffusion



(b) Convert raster image to a vector



(c) VectorFusion: Fine tune by latent score distillation

Style Control – Primitive 제약으로 스타일 선택

Iconography

Primitives	Closed Bézier curves
# Paths	64 paths, 4 segments
Init	Random (scratch) or LIVE
Prompt	"...minimal flat 2d vector icon..."
Detail	paths 수로 조절 (16→64→128)

Pixel Art

Primitives	Square SVG polygons
Grid	32×32
Loss	LSDS + L2 (oversaturation 방지)
Prompt	"...pixel art. trending on artstation"
Issue	SD 직접 변환 → blurry; VF 해결

Sketch

Primitives	Unclosed Bézier (fixed width)
Strokes	16 strokes, 6px, 5 seg
Optimized	Control points only
Init	Random (SD inconsistent)
Prompt	"...minimal 2d line drawing..."

Style Control — Primitive 제약으로 스타일 선택



Figure 6. The number of Bézier paths controls the level of detail in generated vector graphics.

Experimental Setup & Evaluation Metrics

평가 데이터셋

128개 diverse captions

출처: CLIPDraw, ES-CLIP, PartiPrompts, DrawBench, DALL-E 1, DreamFusion

※ ground-truth SVG 없음 → CLIP 기반 자동 평가 지표 사용

Baselines

CLIPDraw: 64 strokes, CLIP score 최적화

Stable Diffusion (raster): K=1/4 rejection sampling

SD init + LIVE: 자동 vectorization (K=1/4)

평가 지표

CLIP Similarity: 생성 이미지 ↔ caption의 CLIP embedding cosine similarity 평균 (×100)

CLIP R-Precision: 128개 caption 중 해당 SVG의 CLIP similarity가 올바른 caption에서 최대인 비율 (%)

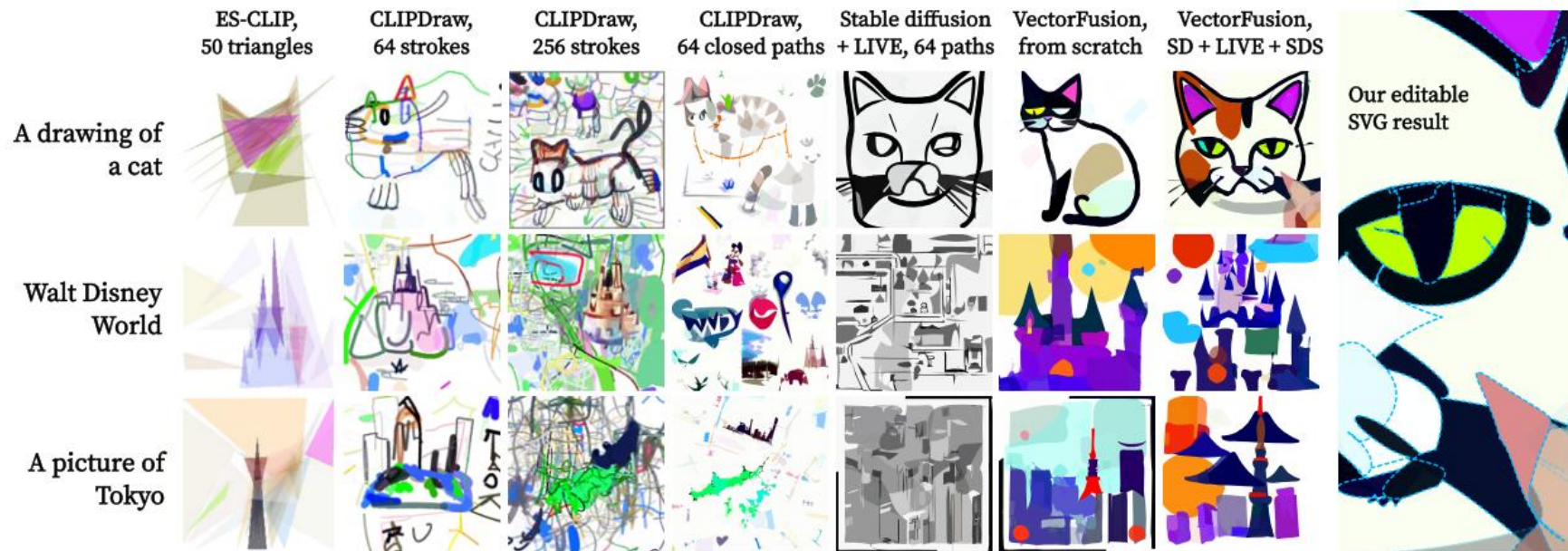
평가 모델: CLIP ViT-L/14 + OpenCLIP ViT-H/14 (이중 평가로 dataset bias 완화)

Quantitative Results – Caption Consistency (64 primitives)

Method	K	CLIP L/14 R-Prec	CLIP L/14 Sim	OpenCLIP H/14 R-Prec	OpenCLIP H/14 Sim
CLIPDraw (scratch)	—	85.2	27.2	77.3	31.7
Stable Diff (raster)	1	67.2	23.0	69.5	26.7
+ rejection sampling	4	81.3	24.1	80.5	28.2
SD init + LIVE	1	57.0	21.7	59.4	25.8
+ rejection sampling	4	69.5	22.9	65.6	27.6
VectorFusion (scratch)	—	76.6	24.3	69.5	28.5
VF + SD init + LIVE	1	78.1	29.1	78.1	29.3
VF + rejection sampling	4	78.9	29.4★	81.3 ★	24.5

Qualitative Results — CLIP vs Diffusion 기반 비교

방법론별 비교 (왼쪽 → 오른쪽으로 품질 향상)



Key finding: Generative diffusion prior(VF 오른쪽 2열)가 discriminative CLIP 최적화 대비 훨씬 coherent-clean한 vector art 생성

Qualitative



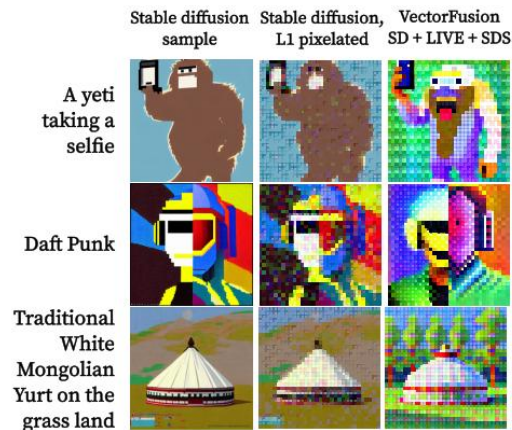
Pixel Art & Sketch Generation

Pixel Art

SD 직접 변환: grid 규칙성 없음 → 32×32에서
blurry·artifact

VectorFusion: square polygon 32×32 그리드 + SDS loss +
L2 (oversaturation 방지)

→ Electric guitar, Daft Punk, hamburger 모두 32×32에서 인식 가능

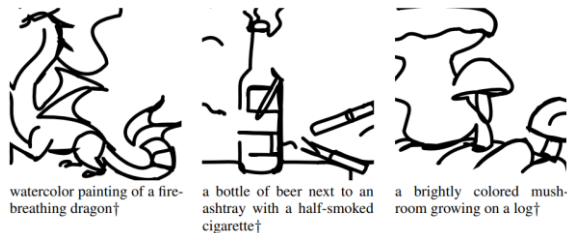


Sketch / Line Drawing

Setup: 16 unclosed Bézier, 6px fixed, control points만 최적화

Key findings:

- reference image 없이 scratch에서 인식 가능한 sketch 생성
- 다중 객체 복잡한 장면 처리 가능
- "watercolor", "brightly colored" 등 무관 용어 무시
- texture 아닌 semantic 정보 포착



Strengths, Limitations & 우리 연구와의 연결점

Strengths

- Captioned SVG 데이터셋 불필요
- Generative prior > discriminative CLIP
- Editable SVG → 기존 design tool 연동
- Icon / Pixel Art / Sketch 지원
- T2I 모델 발전과 함께 자동 개선

Limitations

- T2I forward pass → 계산 비용 높음
- Stable Diffusion dataset bias 상속
- Semantic component 분해 없음
- SVG 개별 최적화 (generative model 아님)
- Global style 하나만 지원

우리 연구 연결점

- DiffVG: 우리 backprop에도 동일 사용
- SDS → T2V 기반 animation loss의 핵심
- Path reinit. → motion delta 안정성

Key takeaway: SDS + differentiable renderer = dataset-free SVG synthesis → 우리 video-diffusion-guided SVG animation framework에 직접 적용 가능

Thank You

For Listening

STUDENT 원 지우 | 20260115 sookmyung university ai vision lab

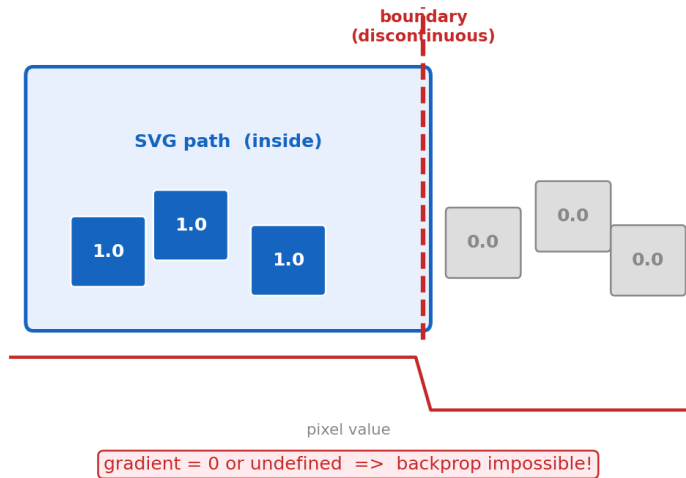
DiffVG – 왜 미분 가능한가?

핵심 아이디어: 경계를 부드럽게 처리

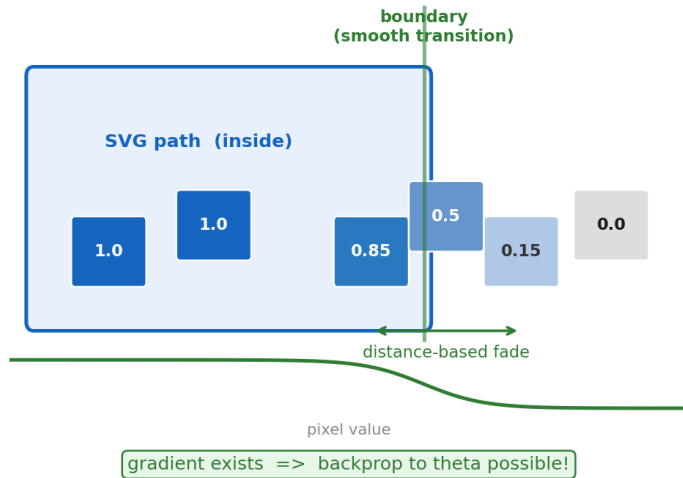
일반 rendering: path 안 = 1.0, 밖 = 0.0 → 불연속 → gradient 없음 → backprop 불가

DiffVG: 경계 근처 픽셀에 거리 기반 부드러운 전환 → 연속 함수 → gradient 존재 → θ 까지 backprop!

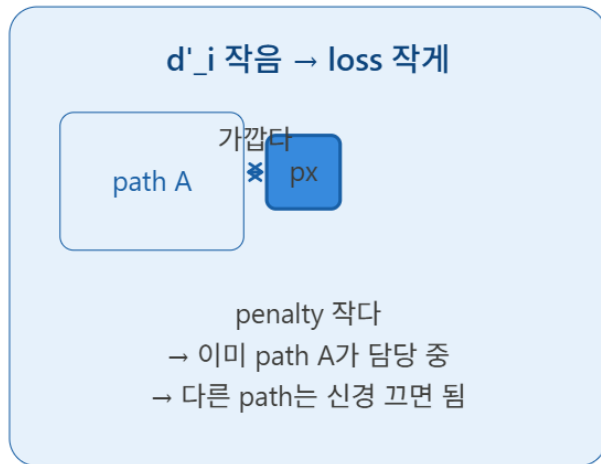
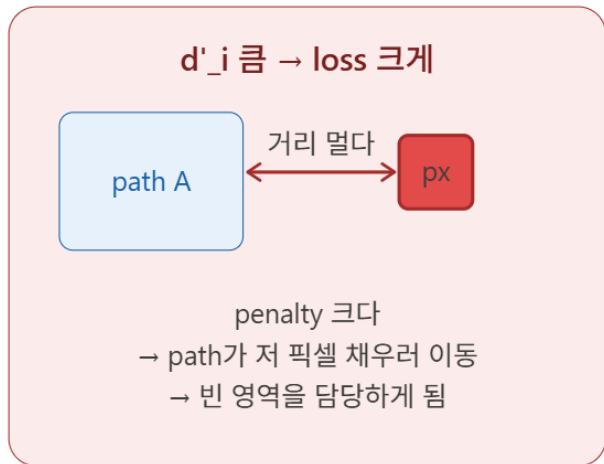
Normal Rendering (no gradient)



DiffVG (differentiable)



UDF Reconstruction Loss (LIVE)



결론: 멀수록 penalty 커서 path가 빈 곳을 채우러 이동
 $\rightarrow$ 자연스럽게 path마다 서로 다른 영역 담당 (1 path = 1 feature)